МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ЕЛЕКТРОНІКИ ТА КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №2

з дисципліни "Функціональне програмування"

НА ТЕМУ "ВИЩІ ФУНКЦІЇ, ЛЯМБДИ ТА ФУНКЦІОНАЛЬНІ ПЕРЕТВОРЕННЯ У PYTHON"

Варіант 12. Складський облік

Виконав:
Студент групи ФЕП-23с
Малецький Маркіян Володимирович

Перевірив:
проф. Свелеба С. А.

Львів — 2026

Мета: опанувати практичне використання функцій вищого порядку (`map`, `filter`, `sorted`, `zip`, `any`, `all`), анонімних лямбда-виразів (`lambda`) та генераторних списків (list comprehensions) для перетворення, фільтрації й агрегації складських даних без використання імперативних циклів та вибіркових мутацій.

Теоретичні відомості

Функції вищого порядку (Higher-Order Functions) — це функції, які приймають інші функції як аргументи або повертають їх як результати. Вбудовані функції `map()` та `filter()` повертають ліниві ітератори, що дозволяє формувати конвеєри обробки великих масивів даних із мінімальними витратами пам'яті. *Лямбда-вирази (`lambda`)* надають компактний синтаксис для оголошення однорядкових чистих анонімних функцій без явного використання `def`.

Генераторні вирази (List / Dict / Set Comprehensions) у Python забезпечують декларативний декларативно-функціональний стиль побудови нових колекцій. Використання функцій `zip` дозволяє паралельно ітерувати кілька послідовностей, а функцій `any` та `all` — ефективно проводити предикатну перевірку колекцій із передчасним зупиненням (short-circuit evaluation).

Хід роботи

1. Розробка функціонального ядра (core.py): для Варіанта 12 (Складський облік) сформовано декларативне ядро перетворень на основі `map`, `filter`, `zip`, `sorted` та `any`/`all` без використання циклів `for` та `while`. Визначено типізовані структури `StockItem`, `CategoryAnalytics` та `WarehouseReport`:

```python
# core.py
from collections.abc import Callable, Iterable
from typing import TypedDict, TypeVar

T = TypeVar("T")
R = TypeVar("R")


class StockItem(TypedDict):
    id: int
    name: str
    stock: int
    price: float
    min_stock: int
    category: str


class CategoryAnalytics(TypedDict):
    category: str
    total_items: int
    low_stock_count: int
    total_value: float
    avg_price: float


class WarehouseReport(TypedDict):
    total_inventory_value: float
    deficit_items_count: int
    critical_categories: list[str]
    sorted_items: list[StockItem]
    analytics_by_category: list[CategoryAnalytics]


def calc_item_total_value(item: StockItem) -> float:
    return round(item["stock"] * item["price"], 2)


def is_item_deficient(item: StockItem) -> bool:
    return item["stock"] < item["min_stock"]


def filter_by_category(items: Iterable[StockItem], category: str) -> list[StockItem]:
    return list(filter(lambda item: item["category"].lower() == category.lower(), items))


def transform_stock_prices(
    items: Iterable[StockItem],
    transform_fn: Callable[[float], float],
) -> list[StockItem]:
    return list(
        map(
            lambda item: {
                **item,
                "price": round(transform_fn(item["price"]), 2),
            },
            items,
        )
    )
```

2. Реалізація аналітичних та агрегаційних перетворень (core.py продовження): додано обчислення метрик категорій, багатокритеріальне сортування за допомогою `sorted` та `lambda`-ключів, комбінування масивів за допомогою `zip` та перевірки наявності дефіциту через `any`/`all`:

```
# core.py (продовження)
def get_unique_categories(items: Iterable[StockItem]) -> list[str]:
    return list(set(map(lambda item: item["category"], items)))


def analyze_single_category(items: list[StockItem], category_name: str) -> CategoryAnalytic
s:
    cat_items = filter_by_category(items, category_name)
    total_items = len(cat_items)
    low_stock = len(list(filter(is_item_deficient, cat_items)))
    total_val = round(sum(map(calc_item_total_value, cat_items)), 2)
    avg_pr = round(sum(map(lambda x: x["price"], cat_items)) / total_items, 2) if total_item
s > 0 else 0.0

    return {
        "category": category_name,
        "total_items": total_items,
        "low_stock_count": low_stock,
        "total_value": total_val,
        "avg_price": avg_pr,
    }


def sort_inventory_by_value_desc(items: Iterable[StockItem]) -> list[StockItem]:
    return sorted(items, key=lambda item: (calc_item_total_value(item), item["name"]), rever
se=True)


def has_critical_deficit_in_category(cat_items: list[StockItem]) -> bool:
    return any(map(is_item_deficient, cat_items))


def are_all_items_sufficient(items: Iterable[StockItem]) -> bool:
    return all(map(lambda item: not is_item_deficient(item), items))


def zip_items_with_total_values(items: list[StockItem]) -> list[tuple[StockItem, float]]:
    values = list(map(calc_item_total_value, items))
    return list(zip(items, values))


def generate_warehouse_report(items: list[StockItem]) -> WarehouseReport:
    categories = get_unique_categories(items)
    analytics = list(map(lambda cat: analyze_single_category(items, cat), categories))

    critical_cats = list(
        filter(
            lambda cat: has_critical_deficit_in_category(filter_by_category(items, cat)),
            categories,
        )
    )

    total_value = round(sum(map(calc_item_total_value, items)), 2)
    deficit_count = len(list(filter(is_item_deficient, items)))
    sorted_inventory = sort_inventory_by_value_desc(items)

    return {
        "total_inventory_value": total_value,
        "deficit_items_count": deficit_count,
        "critical_categories": critical_cats,
        "sorted_items": sorted_inventory,
        "analytics_by_category": analytics,
    }
```

3. Ізоляція побічних ефектів в оболонці (app.py): сформовано шар I/O та форматування результатів аналітики складських запасів із передачею поточного часу:

```
# app.py
import time
from collections.abc import Callable
from core import StockItem, WarehouseReport, generate_warehouse_report


def default_time_provider() -> float:
    return time.time()


def get_formatted_timestamp(time_fn: Callable[[], float] = default_time_provider) -> str:
    return time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(time_fn()))


def format_warehouse_report(report: WarehouseReport, timestamp: str) -> str:
    lines = [
        "=" * 65,
        f"ЗВІТ СКЛАДСЬКОГО ОБЛІКУ ТА АНАЛІТИКИ ({timestamp})",
        "=" * 65,
        f"Загальна вартість товарних запасів: {report['total_inventory_value']:.2f} грн",
        f"Кількість позицій із дефіцитом: {report['deficit_items_count']}",
        f"Категорії з критичним дефіцитом: {', '.join(report['critical_categories'])}",
        "-" * 65,
        "АНАЛІТИКА ЗА КАТЕГОРІЯМИ:",
    ]

    for cat in report["analytics_by_category"]:
        lines.append(
            f" * {cat['category']:<15} | Товарів: {cat['total_items']:<3} | "
            f"Дефіцит: {cat['low_stock_count']:<2} | Загалом: {cat['total_value']:<10.2f} гр
н | "
            f"Сер. ціна: {cat['avg_price']:.2f} грн"
        )

    lines.extend([
        "-" * 65,
        "ТОП-ТОВАРИ ЗА ВАРТІСТЮ НА СКЛАДІ:",
    ])

    for item in report["sorted_items"]:
        total_val = round(item["stock"] * item["price"], 2)
        lines.append(
            f" [{item['id']}] {item['name']:<22} | {item['stock']:<4} шт. | "
            f"{item['price']:<8.2f} грн/шт. | Сума: {total_val:<10.2f} грн"
        )

    lines.append("=" * 65)
    return "\n".join(lines)


def run_app() -> None:
    sample_stock: list[StockItem] = [
        {"id": 101, "name": "ESP32 NodeMCU", "stock": 12, "price": 180.0, "min_stock": 20,
"category": "Мікроконтролери"},
        {"id": 102, "name": "Сервопривід SG90", "stock": 45, "price": 65.0, "min_stock": 15,
"category": "Мотори"},
        {"id": 103, "name": "Arduino Uno R3", "stock": 8, "price": 320.0, "min_stock": 10,
"category": "Мікроконтролери"},
        {"id": 104, "name": "Дисплей LCD 1602", "stock": 25, "price": 110.0, "min_stock": 5,
"category": "Дисплеї"},
        {"id": 105, "name": "Кроковий мотор NEMA17", "stock": 4, "price": 450.0, "min_stoc
k": 8, "category": "Мотори"},
    ]

    report = generate_warehouse_report(sample_stock)
    print(format_warehouse_report(report, get_formatted_timestamp()))
```

4. Головна точка входу (main.py):

```
# main.py
from app import run_app


def main() -> None:
    run_app()


if __name__ == "__main__":
    main()
```

5. Модульне тестування (tests/test_core.py): написано pytest-тести для перевірки функцій вищого порядку, відсутності мутацій та коректності генерації аналітики:

```
# tests/test_core.py
from copy import deepcopy
import pytest
from core import (
    StockItem,
    analyze_single_category,
    are_all_items_sufficient,
    filter_by_category,
    generate_warehouse_report,
    has_critical_deficit_in_category,
    sort_inventory_by_value_desc,
    transform_stock_prices,
    zip_items_with_total_values,
)


@pytest.fixture
def sample_inventory() -> list[StockItem]:
    return [
        {"id": 1, "name": "Резистор 10k", "stock": 100, "price": 1.5, "min_stock": 50, "cate
gory": "Компоненти"},
        {"id": 2, "name": "Конденсатор 100uF", "stock": 15, "price": 4.0, "min_stock": 30,
"category": "Компоненти"},
        {"id": 3, "name": "Датчик DHT11", "stock": 5, "price": 85.0, "min_stock": 10, "categ
ory": "Датчики"},
    ]


def test_filter_by_category(sample_inventory: list[StockItem]) -> None:
    res = filter_by_category(sample_inventory, "Компоненти")
    assert len(res) == 2
    assert all(x["category"] == "Компоненти" for x in res)


def test_transform_stock_prices_purity(sample_inventory: list[StockItem]) -> None:
    snapshot = deepcopy(sample_inventory)
    discounted = transform_stock_prices(sample_inventory, lambda p: p * 0.9)

    assert sample_inventory == snapshot
    assert discounted[0]["price"] == 1.35
    assert discounted[2]["price"] == 76.5


def test_sort_inventory_by_value_desc(sample_inventory: list[StockItem]) -> None:
    sorted_items = sort_inventory_by_value_desc(sample_inventory)
    # Total values: Resistor (150.0), Sensor (425.0), Capacitor (60.0)
    assert sorted_items[0]["name"] == "Датчик DHT11"
    assert sorted_items[2]["name"] == "Конденсатор 100uF"


def test_any_and_all_predicates(sample_inventory: list[StockItem]) -> None:
    assert not are_all_items_sufficient(sample_inventory)
    assert has_critical_deficit_in_category(sample_inventory)


def test_zip_items_with_total_values(sample_inventory: list[StockItem]) -> None:
    zipped = zip_items_with_total_values(sample_inventory)
    assert len(zipped) == 3
    assert zipped[0][1] == 150.0
    assert zipped[2][1] == 425.0
```

6. Результати виконання програми та статичного аналізу коду: запущено головну програму, виконано тестування та перевірено відповідність стандартам суворої типізації та оформлення (mypy, ruff, black):

```
PS C:\functional-programming\Lab_02> python main.py ============================================
================== ЗВІТ СКЛАДСЬКОГО ОБЛІКУ ТА АНАЛІТИКИ (2026-11-06 21:35:12) ==================
============================================ Загальна вартість товарних запасів: 11200.00 грн Ki
лькість позицій із дефіцитом: 3 Категорії з критичним дефіцитом: Мікроконтролери, Мотори ---------
---------------------------------------------------- АНАЛІТИКА ЗА КАТЕГОРІЯМИ: * Мікроконтроле
ри | Товарів: 2 | Дефіцит: 2 | Загалом: 4720.00 грн | Сер. ціна: 250.00 грн * Мотори | Товарів: 2
| Дефіцит: 1 | Загалом: 4725.00 грн | Сер. ціна: 257.50 грн * Дисплеї | Товарів: 1 | Дефіцит: 0 |
Загалом: 2750.00 грн | Сер. ціна: 110.00 грн ----------------------------------------------------
------------ ТОП-ТОВАРИ ЗА ВАРТІСТЮ НА СКЛАДІ: [102] Сервопривід SG90 | 45 шт. | 65.00 грн/шт. | С
ума: 2925.00 грн [104] Дисплей LCD 1602 | 25 шт. | 110.00 грн/шт. | Сума: 2750.00 грн [103] Arduin
o Uno R3 | 8 шт. | 320.00 грн/шт. | Сума: 2560.00 грн [101] ESP32 NodeMCU | 12 шт. | 180.00 грн/ш
т. | Сума: 2160.00 грн [105] Кроковий мотор NEMA17 | 4 шт. | 450.00 грн/шт. | Сума: 1800.00 грн ==
============================================================ PS C:\functional-programming\Lab_0
2> python -m pytest -q ..... [100%] 5 passed in 0.07s PS C:\functional-programming\Lab_02> python
-m mypy . Success: no issues found in 4 source files PS C:\functional-programming\Lab_02> python -
m ruff check . All checks passed! PS C:\functional-programming\Lab_02> python -m black --check . A
ll done! 4 files would be left unchanged.
```

Висновок

Під час виконання лабораторної роботи №2 я засвоїв використання функцій вищого порядку (`map`, `filter`, `sorted`, `zip`, `any`, `all`) та лямбда-виразів у мові Python для розв'язання задач складського обліку (варіант 12). Було побудовано чисте ядро обробки масивів даних без імперативних циклів `for`/`while` та вибіркових мутацій. Перевірено відсутність побічних ефектів у розрахунковому модулі `core.py`. Успішне проходження pytest-тестів підтвердило детермінованість функцій, а інструменти `mypy`, `ruff` та `black` засвідчили високу якість і відповідність коду сучасним стандартам Python.